Manual técnico — REDXela

1. Alcance

REDXela es una aplicación de escritorio académica que simula un centro de distribución. El backend está desarrollado en Java 17; la interfaz está hecha con Swing y el proyecto usa la estructura estándar de Maven bajo com.navi.redxela.

La aplicación modela concurrencia con hilos, semáforos y una cola bloqueante dentro de una JVM. Las áreas operativas son hilos independientes del mismo programa, no procesos del sistema operativo separados. Esta decisión permite mostrar los conceptos de sincronización y comunicación entre áreas sin introducir procesos externos ni dependencias adicionales.

2. Requisitos funcionales cubiertos

Requisito del planteamiento	**Implementación**
Recursos limitados	Recurso define capacidad para empaque (6), carga (2), bodega (10), montacargas (3), calidad (4) y escáner (1).
Cinco niveles de servicio	NivelServicio guarda nombre, orden y objetivo de espera.
Pedidos con productos, cantidades y estado	Pedido, ArticuloPedido, Cliente y EstadoPedido.
Tres políticas de planificación	PoliticaAtencion ordena por servicio, antigüedad o unidades.
Escala configurable del tiempo simulado	TiempoSimulado mantiene el reloj lógico acelerable; la vista Pedidos permite modificar el factor entre ×1 y ×100.
Cinco tipos de mercancía	TipoMercancia asigna una combinación distinta de recursos a cada tipo.
Pedidos automáticos y manuales	MotorDistribucion genera pedidos y la vista Pedidos permite crear otros.
Recepción coordinada con inventario	GestorRecepcion transmite eventos en una cola acotada; un hilo de Inventario los procesa.

Requisito del planteamiento	Implementación
Almacenamiento con varias ubicaciones	InventarioService asigna unidades en 20 ubicaciones y admite lotes repartidos.
Prevención de deadlock normal	GestorRecursos intenta adquirir el conjunto completo sin esperar y revierte una adquisición parcial.
Demostración y resolución de deadlock	ControlConflicto hace que dos hilos bloqueen semáforos en orden inverso y resuelve el ciclo por interrupción controlada.
Interfaz, métricas y manuales	Vistas Swing y los documentos de usuario/técnico.

3. Estructura del proyecto

src/main/java/com/navi/redxela/

```
├── Main.java
├── domain/
│   ├── ArticuloPedido.java
│   ├── Cliente.java
│   ├── EstadoConflicto.java
│   ├── EstadoPedido.java
│   ├── EventoRecepcion.java
│   ├── NivelServicio.java
│   ├── Pedido.java
│   ├── PoliticaAtencion.java
│   ├── Producto.java
│   ├── Recurso.java
│   ├── ResumenProducto.java
│   ├── ResumenRecurso.java
│   └── TipoMercancia.java
├── service/
│   ├── CatalogoProductos.java
│   ├── ControlConflicto.java
│   ├── GestorRecepcion.java
│   ├── GestorRecursos.java
│   ├── InventarioService.java
│   └── MotorDistribucion.java
└── ui/
    ├── Dialogos.java
    ├── MainFrame.java
```

```
├── ModelosTabla.java
├── UIStyles.java
├── VistaConflicto.java
├── VistaDashboard.java
├── VistaInventario.java
└── VistaPedidos.java
```

- **domain:** datos y reglas básicas de pedidos, cuentas, mercancía, recursos y eventos.
- **service:** coordinación de pedidos, inventario, recepciones, semáforos y escenario de deadlock.
- **ui:** ventana principal, vistas Swing, diálogos y modelos de tablas.
- **pom.xml:** Java 17, plugins Maven y clase de entrada para el JAR.

4. Modelo y flujo de datos

4.1 Pedidos y clientes

Cliente asocia una cuenta con un NivelServicio. Pedido contiene una lista inmutable de ArticuloPedido; cada artículo señala un Producto del catálogo y sus unidades. La mercancía y los recursos se derivan del producto para evitar valores contradictorios dentro del pedido.

El estado de un pedido es uno de EN_ESPERA, ESPERANDO_INVENTARIO, ESPERANDO_RECURSOS, EN_PROCESO o COMPLETADO. El identificador se crea al registrar el pedido. Las fechas de creación, inicio y final permiten calcular espera y proceso.

4.2 Catálogo y recursos por mercancía

Mercancía	**Recursos**
Frágil	Calidad + empaque
Pesada	Montacargas + carga
Refrigerada	Bodega + montacargas
Electrónica	Escáner + calidad
Peligrosa	Escáner + calidad + bodega

Las combinaciones son distintas. Electrónica y peligrosa comparten dos recursos, lo que permite observar contención de forma natural en la planificación. Los productos que asigna CatalogoProductos cubren estos tipos: vajilla y lámpara de vidrio (frágiles), refrigeradora y

batería (pesadas), vacunas (refrigeradas), tableta y monitor (electrónicos), y limpiador industrial (peligroso).

Los seis semáforos se inicializan con las capacidades de Recurso: 6 estaciones de empaque, 2 áreas de carga, 10 encargados de bodega, 3 montacargas, 4 estaciones de calidad y 1 escáner. Un pedido obtiene un permiso por cada tipo de recurso requerido, independientemente del número de unidades de ese producto. Solo los pedidos **en proceso** usan permisos; al terminar se liberan. Un pedido que espera inventario o recursos no aparece como usuario activo de esos semáforos. Por eso un recurso puede estar disponible si el generador no ha elegido recientemente productos que lo necesiten o si esos productos se quedaron sin stock.

4.3 Inventario físico y reserva

InventarioService mantiene por producto las unidades disponibles y las cantidades situadas en ubicaciones físicas. El almacén tiene 20 ubicaciones con capacidad distribuida según la capacidad total. La asignación recorre las ubicaciones con un salto de siete posiciones; de esta manera, un lote grande puede quedar repartido y no contiguo. Una ubicación puede alojar varios productos, por lo que el mismo código puede aparecer en distintas filas de producto; dentro de una fila no se repite.

Al despachar un pedido:

1. reservar valida todas las cantidades bajo el monitor del inventario.
2. Las unidades pasan de disponibles a comprometidas para pedidos en proceso; siguen ocupando espacio físico mientras el pedido se procesa. La columna **En proceso** refleja esta cantidad.
3. Si el procesamiento termina, confirmarDespacho retira las unidades de sus ubicaciones y libera espacio.
4. Si no se pueden asignar recursos o el procesamiento se interrumpe, liberar devuelve las cantidades a disponibles.

La columna **En proceso** del resumen de inventario es unidades físicas del producto − unidades disponibles. En el flujo normal equivale a la cantidad retenida por pedidos activos: no significa una recepción nueva ni un segundo lote físico.

El estado inicial carga 25 unidades por cada uno de los ocho productos (200 unidades en total). La ocupación global se calcula sumando las ocupaciones físicas de las ubicaciones para mantener el total alineado con el detalle. Las recepciones calculan primero un plan completo de ubicaciones y solo lo aplican si cabe toda la solicitud; si no, se rechaza sin modificar ninguna ubicación. La capacidad se configura con -Dredxela.capacidad=...; el valor predeterminado es 500 y el mínimo es 20.

5. Planificación de pedidos

MotorDistribucion inicia un planificador de un solo hilo que revisa la cola cada 200 ms. Para cada candidato, en el orden de la política vigente:

1. Comprueba que haya inventario suficiente.
2. Reserva las cantidades requeridas.
3. Calcula la unión de los recursos de los tipos de mercancía del pedido.
4. Intenta adquirir todos los recursos.
5. Si no están disponibles, devuelve inmediatamente el inventario reservado y deja el pedido en espera.
6. Si se obtienen, marca el pedido en proceso y lo asigna al grupo de seis trabajadores.
7. El tiempo simulado se estima con 15 s + 20 s × unidades, con máximo de 300 segundos; el reloj acelerado escala el tiempo observado.
8. Al finalizar, confirma el despacho y libera los recursos.

El tamaño del pedido determina el trabajo estimado. El generador crea pedidos cada segundo, rota entre los cinco clientes de ejemplo y solicita entre 2 y 8 unidades para hacer más visible la contención. A ×1, pedidos de 2 a 8 unidades tardan aproximadamente 55 a 175 segundos; un factor ×10 los reduce a 5.5 a 17.5 segundos reales. Cada trabajador mantiene el pedido activo y sus semáforos mientras duerme en intervalos cortos y actualiza el progreso: representa tiempo de ocupación de las estaciones, no cálculo intensivo artificial de CPU. Pedidos independientes avanzan en paralelo hasta que se agote algún recurso.

Políticas

- **Prioridad estricta:** orden ascendente por nivel numérico y luego fecha de creación.
- **Prioridad con antigüedad:** puntaje nivel × 5 − minutosEsperando; un pedido más antiguo puede superar a niveles inferiores con el tiempo.
- **Menos unidades primero:** cantidad total ascendente, luego nivel y fecha.

La política solo ordena pedidos que aún esperan. No cambia recursos ni interrumpe pedidos que ya comenzaron.

Velocidad del reloj simulado

TiempoSimulado acumula tiempo monotónico real ponderado por el factor activo. ×1 es tiempo real; con ×2 un intervalo real de 60 segundos avanza 120 segundos simulados. El acelerador se puede modificar en ejecución desde **Pedidos** o al iniciar con -Dredxela.acelerador=2 (rango ×1–×100). Fechas de pedidos, antigüedad, métricas y progreso del trabajo utilizan el tiempo simulado. El factor se conserva como configuración global durante la sesión, no como dato del pedido.

6. Sincronización y comunicación

6.1 Recursos y exclusión mutua

GestorRecursos mantiene un Semaphore justo por tipo de recurso, inicializado con su capacidad. La adquisición normal usa tryAcquire y no queda esperando mientras retiene una parte del conjunto: si un recurso falla, libera inmediatamente cualquier permiso tomado durante ese intento. Así se evita la condición de espera circular en el planificador normal y no se mantienen reservas parciales entre ciclos.

Los mapas de ocupación se protegen con métodos synchronized. El Semaphore limita el número de usuarios simultáneos; el monitor coordina la modificación de los registros de quién tiene cada permiso.

6.2 Comunicación recepción → inventario

GestorRecepcion tiene una ArrayBlockingQueue de 100 eventos. El hilo de Recepción produce solicitudes y el hilo redxela-area-inventario consume una por una. La cola tiene capacidad finita: una solicitud se acepta solo si pudo insertarse; si está llena, la interfaz avisa y permite reintentar. No se descarta silenciosamente una recepción aceptada ni se bloquea la interfaz esperando espacio.

Después de actualizar existencias, Inventario notifica al motor. El planificador revisa periódicamente los pedidos en espera para reintentar los que ya cuentan con stock.

6.3 Hilos principales

Hilo o grupo	**Responsabilidad**
EDT de Swing	Dibuja la interfaz y atiende eventos de usuario.
redxela-planificador	Revisa la cola y asigna los pedidos elegibles.
Seis redxela-pedido	Simulan el trabajo de pedidos en paralelo.
redxela-generador	Produce llegadas automáticas mientras está activado.
redxela-area-inventario	Consume eventos de recepción y actualiza el almacén.
redxela-conflicto-a/b	Retienen y solicitan recursos en orden inverso en el escenario de deadlock.

Las vistas consultan snapshots durante el refresco de Swing y las notificaciones de servicios se envían al EDT con SwingUtilities.invokeLater.

7. Escenario de interbloqueo

ControlConflicto usa el mismo GestorRecursos que los pedidos normales. Para que el conflicto sea reproducible, la operación A retiene el único escáner y la B toma las cuatro estaciones de calidad. A solicita calidad y B solicita el escáner. Los dos hilos usan Semaphore.acquire() y quedan bloqueados en un ciclo real de espera.

El escenario solo se prepara si puede reservar ambos recursos requeridos. Si están ocupados, la interfaz informa el motivo y se puede volver a intentar después.

Al elegir un ganador, el controlador interrumpe el hilo descartado mientras espera su segundo recurso. El finally del hilo libera sus recursos iniciales; el hilo ganador obtiene el recurso liberado, termina su trabajo simulado y suelta todos sus permisos. La interfaz conserva el resultado y marca el otro trabajo para volver a espera. Los pedidos normales no usan esta estrategia de espera parcial; el escenario existe para demostrar el problema y su resolución.

8. Interfaz y métricas

MainFrame aloja cuatro vistas con CardLayout. Las vistas refrescan sus modelos de tabla desde Swing. ModeloPedidos y sus variantes presentan estados, progreso y tiempo restante; ModeloRecursos, ModeloInventario y ModeloRecepciones convierten snapshots del dominio en filas de tabla.

Las métricas se calculan en memoria:

- pedidos completados: contador incrementado al terminar un pedido;
- espera promedio: media del tiempo entre creación e inicio, para pedidos completados;
- proceso promedio: media entre inicio y despacho;
- ocupación: permisos usados divididos por la suma de capacidades de los recursos.

9. Manejo de errores y ciclo de vida

- Se valida nombre de cuenta, artículos, cantidades, existencias y capacidad antes de modificar el estado correspondiente.
- Las recepciones que no caben se conservan en el historial como rechazadas con una explicación.
- Los errores de una llegada generada o un listener no detienen el planificador ni al resto de la interfaz.
- Al cerrar la ventana se detiene el temporizador de Swing, se interrumpen los hilos de conflicto y recepción y se apagan los ejecutores.

- La información no persiste en disco; es una decisión para mantener el alcance de la simulación enfocado en concurrencia y recursos.

10. Construcción y ejecución

mvn clean package

java -jar target/redxela-1.0.0.jar

En IntelliJ, abre pom.xml, selecciona JDK 17 y ejecuta com.navi.redxela.Main. Para variar el almacén, agrega una opción como -Dredxela.capacidad=800 en VM options.

11. Validación manual recomendada

1. Crear pedidos de dos niveles diferentes y verificar el orden de espera con cada política.
2. Crear un pedido con varias unidades y observar progreso y tiempo estimado.
3. Crear un pedido sin existencias para comprobar Esperando inventario; luego recibir el producto y verificar que se procese.
4. Recibir una cantidad que exceda el espacio libre y comprobar el rechazo sin alterar el stock.
5. Iniciar el generador, detenerlo y confirmar que los pedidos existentes continúan.
6. Iniciar el conflicto cuando los recursos estén libres; seleccionar A y repetir seleccionando B.
7. Revisar que al terminar el conflicto los permisos vuelvan a estar libres y la simulación continúe.